What Else Needs Fixing?

Exploring Cost-Effective Test-Time Compute for Revision Propagation in Artifacts Generated Through Conversation

Daisuke Kikuta

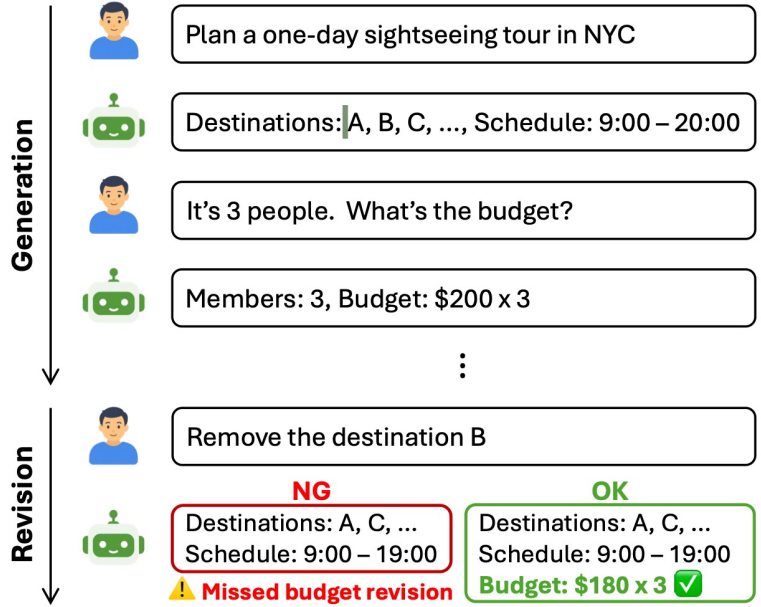
NTT, Inc.

© NTT, Inc. 2026

Background

- LLMs often help users generate artifacts through **iterative cycles of generation and revision in conversation**

- A challenge here is that, **when users specify only a local change, LLMs must identify dependencies and update all affected parts ($\stackrel{\text{def}}{=}$ Revison Propagation)**



© NTT, Inc. 2026

Related Works

Repository-level coding[1][2][3]

Given an edit request, LLMs must locate relevant files and propagate edits across codebase-wide dependencies of classes, functions, variables, etc.

Document editing[4][5]

Given an edit request, LLMs must update dependent document elements (e.g., figures, sections, word choices, etc.) or claims to preserve consistency

These exisiting works focus on:
- Artifacts whose dependencies are largely explicit or statically analyzable
- Only final artifacts (without conversation history before they were generated)

In contrast, we focus on cases where the **dependencies are more implicit** and **the dependencies and their semantic are included in conversation**

© NTT, Inc. 2026

[1] Jimenez et al., ICLR 2024 [2] Bairi et al., FSE 2024 [3] Du et al., ACL 2025
[4] Wang et al., ACL 2026 [5] Kruthof, arXiv 2026

Contributions

Our contributions are twofold:

■ New benchmark

> We propose RevPropBench, which evaluates whether LLMs can patch all and only the dependent parts of JSON artifacts, given the conversation history and local revision request

> We evaluate nine revision methods with six LLMs on the benchmark

■ Cost analysis of test-time compute

> We analyze performance scaling by increasing the number of test-time samples

> We provide the most cost-effective sample size for test-time compute in our problem

© NTT, Inc. 2026

Proposed Benchmark: RevPropBench

Data generation & evaluation

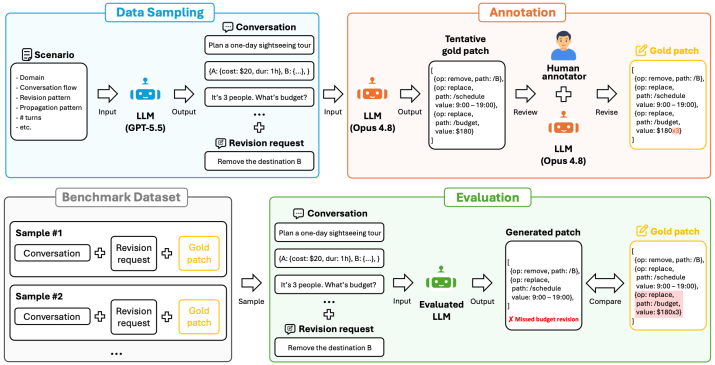
Generate the pairs of a conversation hisotry and a local revision request using LLMs

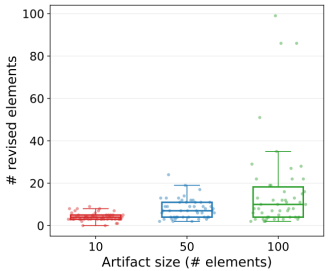
→ LLM-assisted human annotation → Compare a LLM-generated patch with the gold patch

Statistics

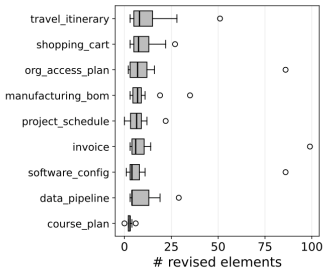
150 samples (dev: 30, test: 120), 9 practical domains, 6 propagation patterns,

3 artifact sizes (10/50/100 elements)

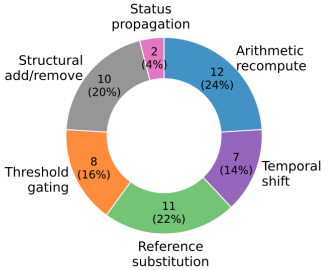




(a) Propagation size by artifact size.



(b) Propagation size by domain.



(c) Propagation patterns.

© NTT, Inc. 2026

Example

Conversation history (H)


I'm planning a trip that begins on May 14, 2026.


start_date: 2026-05-14


Plan for me to fly out two days after the trip begins.


outbound_flight_date: 2026-05-16


Check into the hotel three days after the trip begins.


hotel_check_in_date: 2026-05-17

⋮

Revision request


Please change the trip to begin on May 17, 2026.

Final JSON artifact

"start_date": "2026-05-14"
"outbound_flight_date": "2026-05-16"
"hotel_check_in_date": "2026-05-17"
"return_flight_date": "2026-05-22"
…

⚠ **Dates look independent in J alone**

Correct revision

"start_date": "2026-05-17"
"outbound_flight_date": "2026-05-19"
"hotel_check_in_date": "2026-05-20"
"return_flight_date": "2026-05-25"
All derived dates shift by +3 days ✅

© NTT, Inc. 2026

Experimental Settings

Models

> **gpt-oss-20b/120b** (reasoning_effort=mid, temperature=0.6)

> **gpt-5.4-mini** (reasoning_effort=mid)

> **qwen-3.5-9b/27b/122b-a10b** (reasoning=true, temperature=0.6)

Revision methods

> **Baselines**

- **J**: Final JSON + request
- **H**: Conversation history + request
- **J+H**: Final JSON + conversation history + request

> **Test-time compute methods** (All candidates are sampled by J+H)

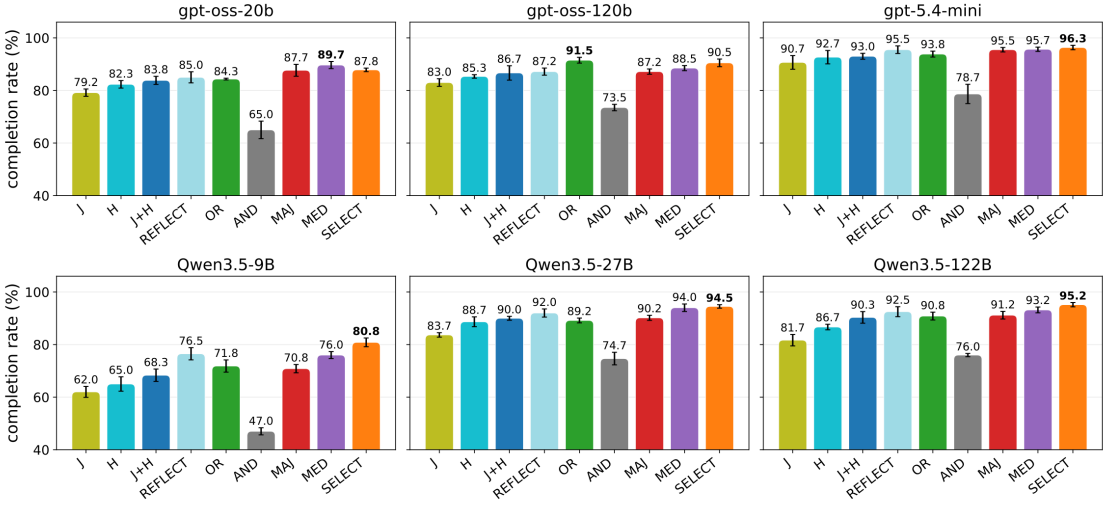
- **REFLECT**: Sequential self-reflection
- **OR/AND/MAJ**: Element-wise merge of parallel samples by or/and/majority
- **MED**: Select medoid candidate by leaf-level agreement
- **SELECT**: LLM chooses the best sampled candidate

We measure the exact-match rate*
between the generated patch and gold patch
*We call it *completion rate*

© NTT, Inc. 2026

Results

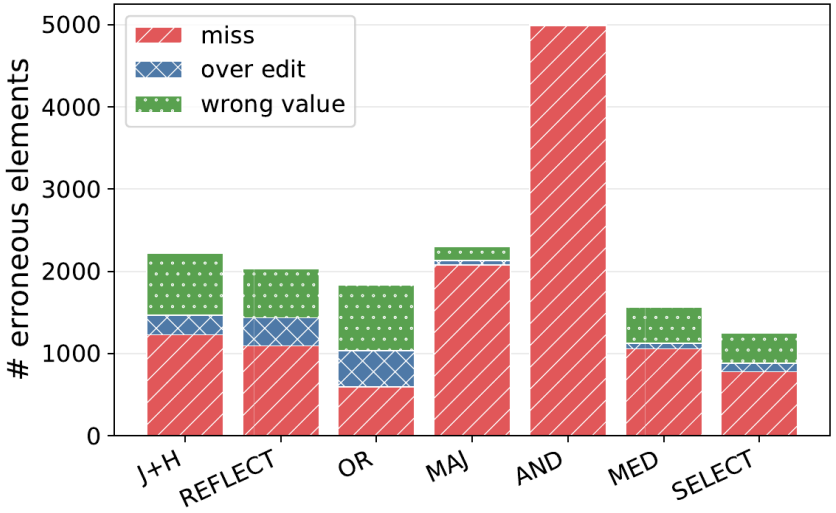
- Baselines achive completion rates of 68.3-93%
- Conversation history is critical for capturing conversationally establisted dependencies and element semantics (J < H < J+H)
- 1st: SELECT improves by 3.3-12.5pp, 2nd: MED improves by 1.8-7.7pp over J+H
  *Each test-time compute methods use 5 LLM calls



© NTT, Inc. 2026

Failure Analysis

- **Error types**
  miss: omitted necessary edits
  over edit: unnecessary edits
  wrong value: incorrect values in necessary edits
- **Misses dominate**: models tend to propagate too little rather than too much
- **AND** fails almost only by misses; **OR** adds many over edits
- **MED** reduces outlier over edits; **SELECT** recovers more misses than MED

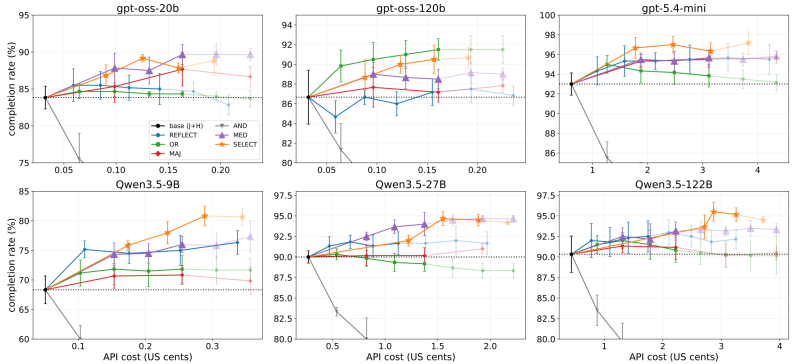


Element-level failures pooled over 6 models (3,600 samples)

© NTT, Inc. 2026

Cost Analysis

- We extend the number of samples and measure how performance scales
- Performance begins to **saturate at around 4-5 LLM calls**
- **3 samples are the most cost-effective** for SELECT and MED (+1 selection call for SELECT)
- SELECT increases latency by 2-6x. **If latency matters, MED offers a better trade-off**, with only ~1.4x latency and ~1.4pp lower performance



Model	J+H	REFLECT				MED				SELECT		
		k=2	k=3	k=4	k=5	k=2	k=3	k=4	k=5	k=3	k=4	k=5
gpt-oss-20b	15.9 s	2.71×	3.37×	3.98×	4.56×	1.21×	**1.36**×	1.44×	1.49×	2.95×	**2.87**×	3.08×
gpt-oss-120b	19.3 s	2.09×	2.76×	3.41×	4.04×	1.08×	**1.12**×	1.15×	1.17×	1.95×	**2.13**×	2.20×
gpt-5.4-mini	7.3 s	2.04×	2.54×	3.03×	3.52×	1.20×	**1.27**×	1.33×	1.39×	2.08×	**2.23**×	2.41×
qwen3.5-9b	23.3 s	2.73×	3.75×	4.75×	5.70×	1.14×	**1.20**×	1.29×	1.32×	3.65×	**4.11**×	4.41×
qwen3.5-27b	58.5 s	2.64×	3.45×	4.24×	5.04×	1.13×	**1.24**×	1.28×	1.35×	4.81×	**5.20**×	5.56×
qwen3.5-122b	41.5 s	3.28×	4.26×	5.14×	6.02×	1.19×	**1.29**×	1.41×	1.47×	6.24×	**5.62**×	5.51×

© NTT, Inc. 2026

Limitations

Scope

Does not target settings where dependency information is available in advance
(e.g., code graphs, knowledge graphs, document structure), as studied in prior work

Gap from real-world conversations

Conversations are synthetic and dependencies are controlled to be deterministic;
ambiguous cases where people may disagree on what to revise are not covered

Performance saturation

gpt-5.4-mini already reaches 93% → we release the **sample generation & annotation tool**
so that the benchmark can be continuously updated

© NTT, Inc. 2026

Conclusion

- We proposed a new benchmark for evaluating the ability of LLMs to propagate revisions across dependent elements in conversationally generated JSON artifacts
- We evaluated nine revision methods with six LLMs on the benchmark, providing practical guidance for cost-effective method selection

Takeaways

- Conversation history is critical for capturing conversationally establisted dependencies and element semantics
- Test-time compute improves revision propagation accuracy, but performance begins to saturate at around 4-5 LLM calls
- SELECT or MED with 3 samples is the most cost-effective

Code & Dataset

© NTT, Inc. 2026